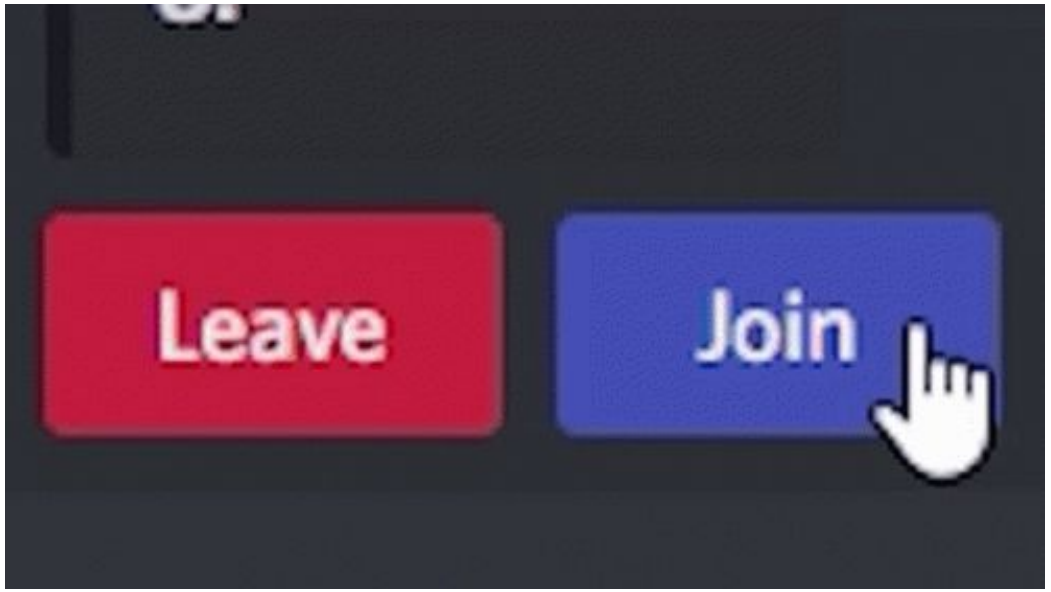
Newton School of Technology

15

Managing State

Join the Lecture on NS Portal :)

Newton School of Technology



Newton School of Technology

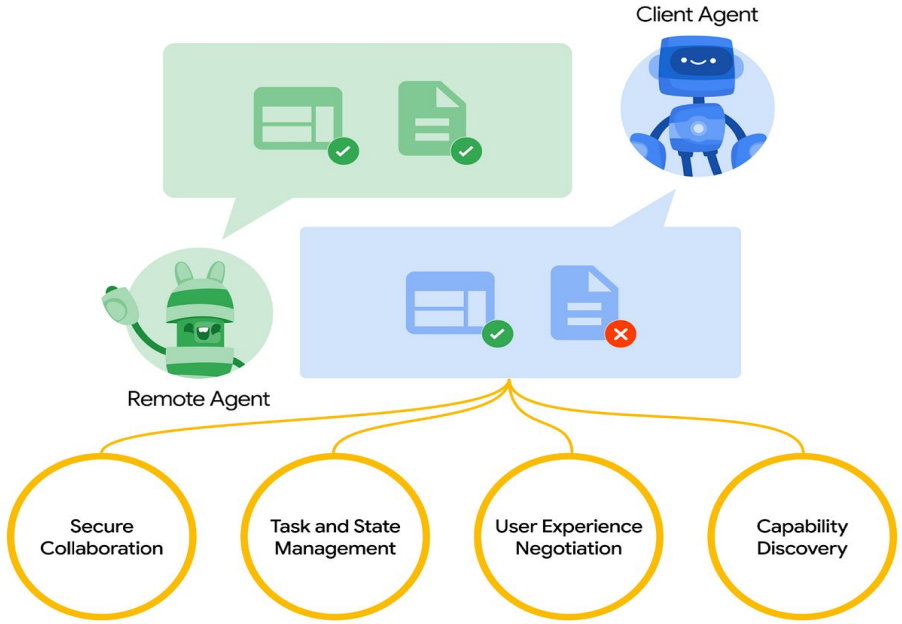
Join the lecture online on your dashboard.

State Management in AI Agents

Newton School of Technology

State management is what allows an AI agent to remember past steps, understand new inputs correctly, and continue long workflows without losing context.

- Maintains the agent's memory of prior interactions so it can make informed decisions.
- Preserves continuity in multi-step tasks, preventing workflow breaks.
- Tracks past inputs, intermediate results, and tool calls to avoid redundancy.
- Prevents fragmented responses by ensuring context is not lost between messages.
- Enables dynamic, context-aware, and predictable behavior in real-world AI workflows.



Why do you think an AI agent loses context without proper state management?

Why State Management Is Required

Newton School of Technology

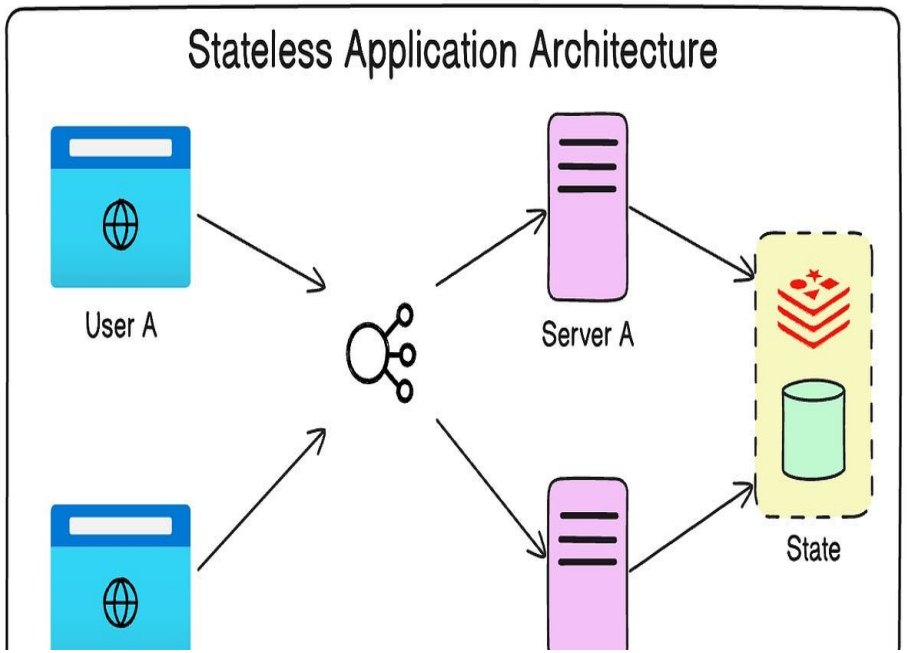
State management is crucial because AI agents need to retain past messages, tool outputs, and workflow progress to act coherently. Without this continuity, the agent loses context and cannot complete tasks accurately.

- Without state, the agent treats every input as new and loses important past instructions.
- Multi-step workflows break because continuity and prior context are missing.
- The agent becomes inconsistent, unable to recall earlier reasoning or constraints.
- Missing tool outputs force unnecessary recomputation and wasted resources.
- Proper state improves accuracy, consistency, and reliability across long tasks.

Problems Faced Without State

Newton School of Technology

- Responses become disconnected when the agent cannot remember earlier instructions or context.
- Multi-step tasks fail because each step is handled independently without past memory.
- Users must repeat information, reducing efficiency and overall experience.
- Logical consistency breaks since decisions cannot reference prior reasoning or preferences.
- The agent re-triggers tools or repeats steps, leading to unreliable and unpredictable behavior.



Understanding State

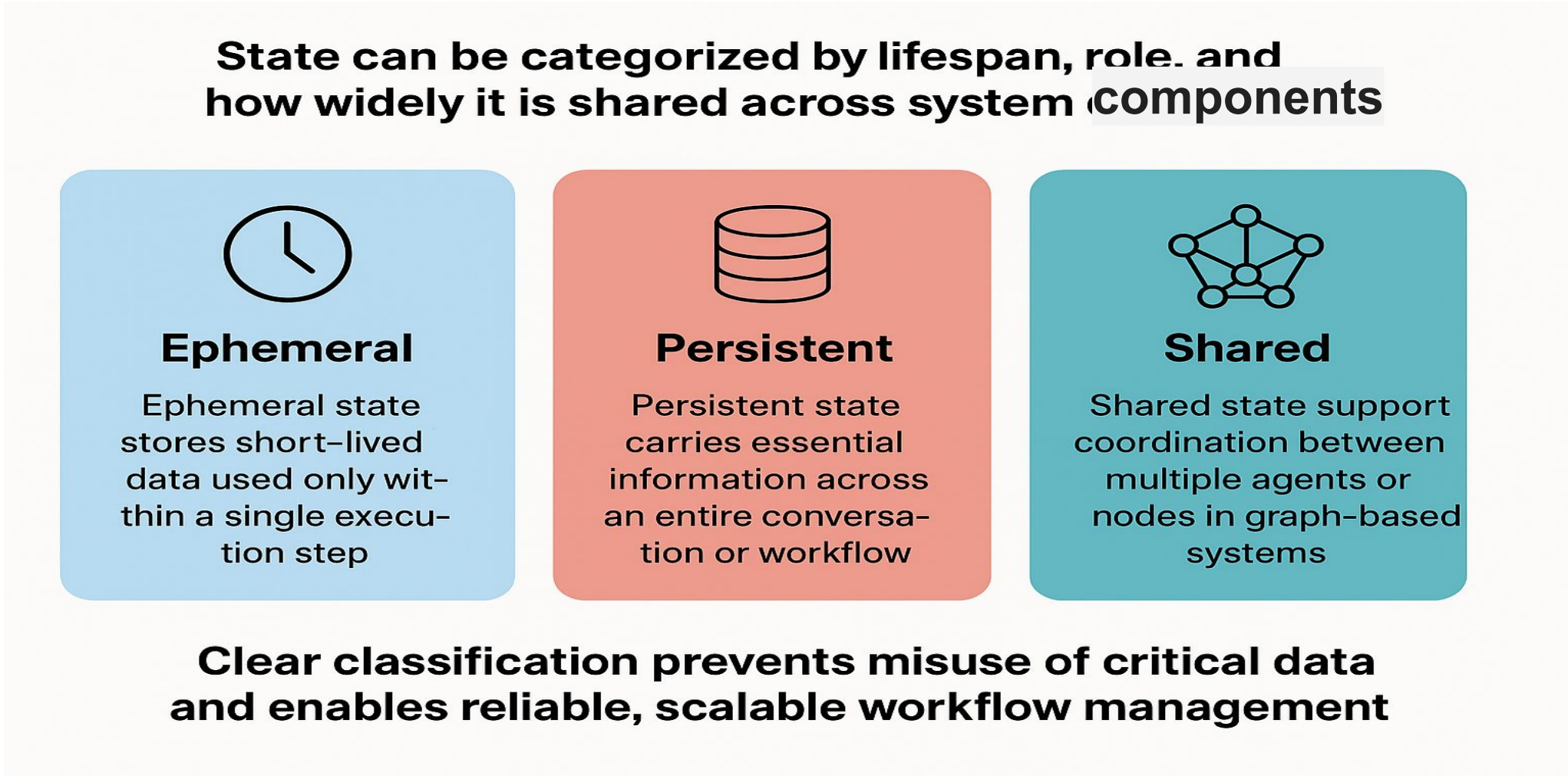
Newton School of Technology

State is the evolving internal snapshot of everything the agent knows at a given moment, updated after each interaction or operation.

- State captures all accumulated knowledge—messages, tool outputs, reasoning steps, and workflow metadata.
- It allows the agent to interpret new instructions in context and evolve its behavior after every interaction.
- State includes both short-term variables and long-term preferences depending on the architecture.
- It tracks completed steps and pending actions, enabling smooth multi-step workflows.
- Well-structured state ensures efficient, reliable, and consistent decision-making across the agent's lifecycle.

Types of State: Overview

Newton School of Technology

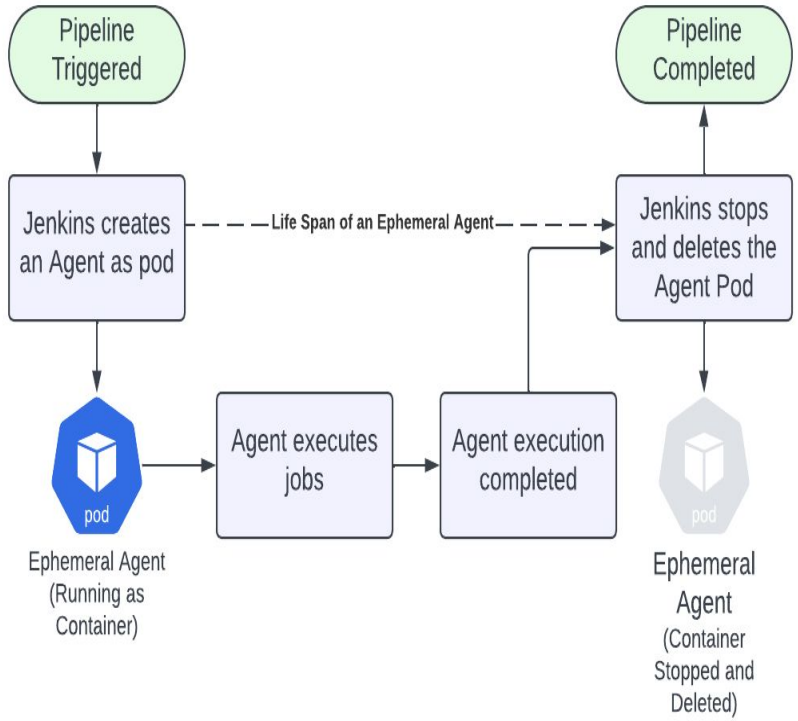


Should ephemeral state store conversation history? Why or why not?

Ephemeral State

Newton School of Technology

- Ephemeral state holds temporary variables or outputs generated during a single computation.
- It may include tool outputs, intermediate LLM reasoning, or one-step calculations.
- This state exists only during execution and is deleted afterward, reducing memory load.
- Ephemeral state is essential for fast, isolated decision-making without polluting long-term context.
- It is ideal for data that will not influence future steps once consumed.
- Managing ephemeral state prevents unnecessary bloating of persistent memory.
- Used heavily in tools, validators, preprocessors, and computational nodes.



Persistent State

Newton School of Technology

- Persistent state stores all crucial information that must survive across multiple steps.
- It includes conversation history, user preferences, validated data, and completed subtasks.
- Persistent state ensures continuity in long-running conversations or workflows.
- It enables the agent to recover from interruptions by reloading previous checkpoints.
- It supports predictability, allowing reasoning to build upon previous actions.
- Persistent state is commonly stored using databases, checkpointers, or memory modules.
- It forms the basis for reproducible and deterministic agent behavior.

Why is persistent state crucial for multi-step reasoning tasks?

Shared State

Newton School of Technology

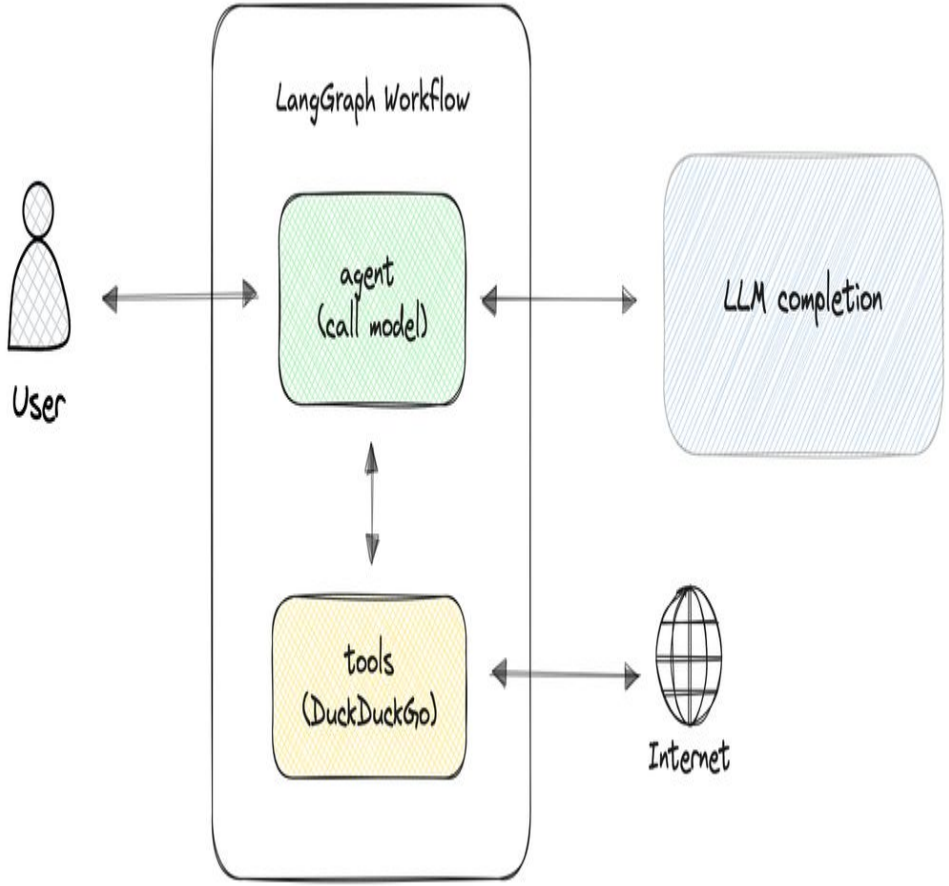
- Shared state is used when multiple nodes or agents contribute to a common workflow.
- It acts as a unified dictionary accessible across components.
- Shared state prevents information silos by allowing nodes to exchange updates efficiently.
- It is crucial for multi-agent collaboration where decisions depend on shared context.
- LangGraph uses shared state to propagate messages and outputs across nodes.
- Proper synchronization prevents conflicts, overwriting, or inconsistent state updates.
- Shared state improves scalability in graph-based workflows involving parallel execution.

Why is persistent state more important for long-running workflows compared to ephemeral state? Provide an example where persistent state prevents system failure.

LangGraph: Why It Is Needed

Newton School of Technology

- LangGraph provides a structured way to manage agent workflows through states, nodes, and edges.
- It supports deterministic execution by defining flow control explicitly.
- LangGraph ensures that state updates are consistent at each step.
- It enables both sequential and parallel execution paths.
- The graph-based approach makes debugging and scaling easier.
- It introduces typed state, reducing accidental key errors or mismatched structures.
- Checkpointing ensures that workflows can resume even after external failures.



Why is a graph-based approach better than traditional sequential code for AI agents?

LangGraph Core Component: State

Newton School of Technology

- In LangGraph, state is defined as a typed Python dictionary representing workflow data.
- It stores messages, tool outputs, progress markers, and any other contextual elements.
- TypedDict ensures strict structure and prevents unexpected mutations.
- State updates happen incrementally after each node execution.
- add_messages behavior ensures conversation continuity without overwriting previous entries.
- State can be deeply nested to support complex multi-agent or multi-tool workflows.
- Persistent state is managed through pluggable checkpointers.

Why might replacing the state entirely (instead of updating it) create problems in multi-step workflows?

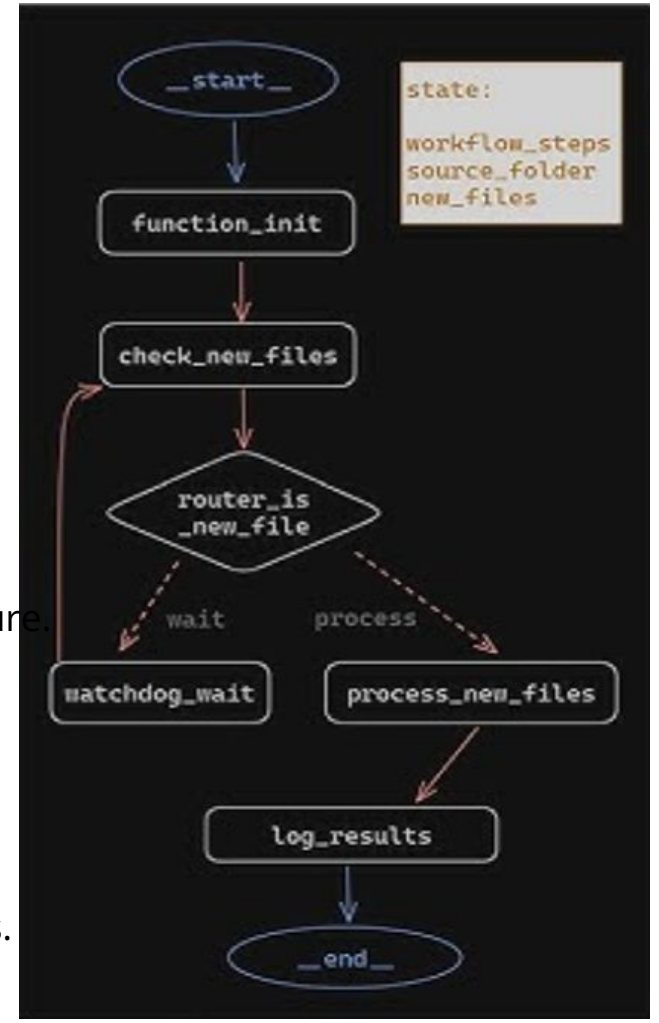
Defining State

Newton School of Technology

```
# Define a Langchain graph
workflow = StateGraph(State)

workflow.add_node("init", function_init)
workflow.add_edge('init', 'check_new_files')

workflow.add_node("check_new_files", check_new_files)
workflow.add_node("watchdog_wait", watchdog_wait)
workflow.add_node("process_new_files", process_new_files)
```



- This TypedDict defines the structure of the agent's state container.
- messages is annotated with add_messages, enabling automatic appending.
- Typed definitions reduce ambiguity and make it easier to reason about workflow structure.
- Structured state prevents accidental overwriting of message history.
- The annotation modifies how state updates are applied.
- This form scales easily when adding new elements such as metadata or checkpoint flags.
- It forms the foundation for all graph operations handled by LangGraph.

LangGraph Core Component: Node

Newton School of Technology

- A node is a Python function that performs a logical action within the workflow.
- Nodes always receive the current state as input and return an updated version.
- They may call LLMs, tools, external APIs, or run local computation.
- Nodes help modularize workflows by separating tasks cleanly.
- They support parallel execution if their incoming edges permit.
- Node behavior is deterministic, ensuring that given the same state, they produce the same output.
- Nodes encapsulate both reasoning steps and operational actions.

Why do you think LangGraph uses multiple small nodes instead of one huge function?

Node Example (Chatbot Node)

Newton School of Technology

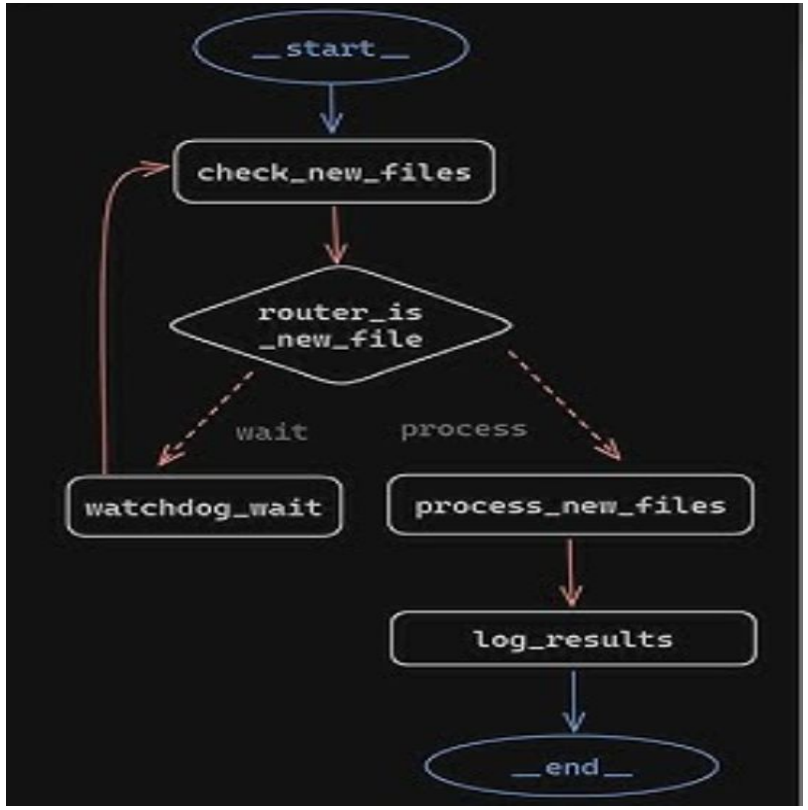
```
def chatbot(state: State):
    messages = state["messages"]
    response = model.invoke(messages)
    return {
        "messages": messages + [AIMessage(content=response.content)]
    }
```

- The node reads the conversation history from state["messages"].
- It passes the messages to an LLM using model.invoke.
- The model returns a response containing generated content.
- The node appends this output as a new AIMessage to the message list.
- State mutation is controlled, structured, and deterministic.
- This design maintains full conversational context while adding new reasoning.

LangGraph Core Component: Edge

Newton School of Technology

- Edges determine which node should execute next based on workflow design.
- Normal edges always transition from one node to another.
- Conditional edges evaluate state to determine the appropriate next step.
- Edges introduce autonomy by allowing dynamic routing paths.
- They enable scalable multi-step workflows.
- Workflows become predictable since routing logic is explicitly defined.
- Edges are essential for orchestrating complex decision-making.



How do conditional edges help an AI agent behave more intelligently?

Building the Graph

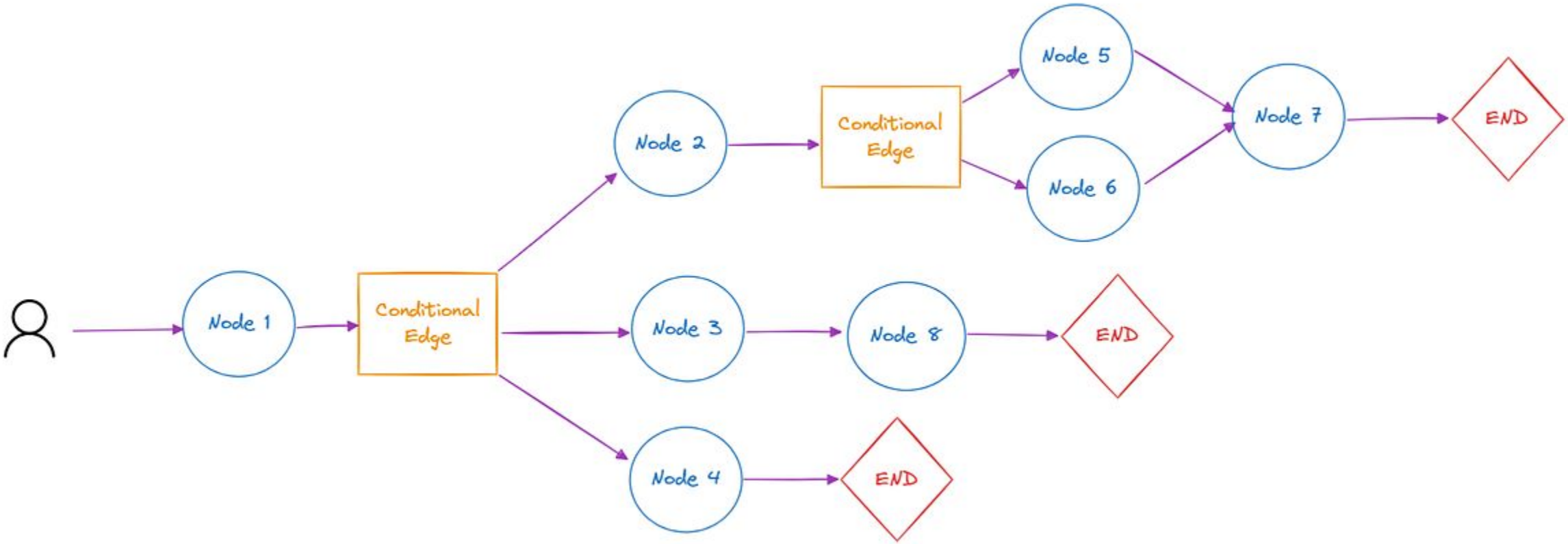
Newton School of Technology

```
graph_builder = StateGraph(State)
graph_builder.add_node("chatbot", chatbot)
graph_builder.add_edge(START, "chatbot")
```

- StateGraph initializes a new graph using the defined State schema.
- Nodes are registered into the graph with unique identifiers.
- Edges establish execution order starting from the START token.
- This simple graph structure already supports full conversational behavior.
- The system ensures deterministic execution by following edges explicitly.
- Additional nodes can be added for tools, validation, or branching.
- The graph can be extended into multi-node autonomous workflows.

Agent workflows using langgraph example

Newton School of Technology



Persistent Memory / Checkpointing

Newton School of Technology

```
memory = MemorySaver()
app = graph_builder.compile(checkpointer=memory)
```

- Memory Saver stores checkpoints of the state across executions.
- It ensures that multi-turn interactions persist even after the application restarts.
- Checkpointers help the agent recover gracefully from system failures.
- They preserve essential data such as messages, tool outputs, and workflow markers.
- Persistent memory enables long-term autonomy.
- It supports debugging by allowing developers to inspect state snapshots.
- Checkpointing avoids recomputing previously completed work.

Runtime Loop Execution

Newton School of Technology

```
while True:
    user_input = input("You: ")
    state["messages"].append(HumanMessage(content=user_input))
    state = app.invoke(state)
    print("AI:", state["messages"][-1].content)
```

- This loop continuously accepts user input to feed the agent.
- Each message is appended to the persistent conversation history.
- app.invoke() processes the current state through the graph.
- The updated state includes the new AI response.
- This simple loop enables a fully functional conversational agent.
- The system updates deterministically based on previous context.
- It demonstrates the practical power of LangGraph for real-time applications.

Task

Newton School of Technology

How does checkpointing improve reliability in multi-turn AI agent workflows? Provide a real-world scenario where checkpoints prevent repeated work.

Advantages of State Management

Newton School of Technology

- Ensures consistent multi-turn conversations by preserving context.
- Enables long-running workflows with multiple steps and checkpoints.
- Supports parallel execution by coordinating shared state across nodes.
- Reduces LLM hallucinations by grounding responses in persistent data.
- Improves predictability so the agent behaves deterministically.
- Supports tool orchestration by preserving intermediate results.
- Enables autonomy by allowing dynamic routing using state information.

Newton School of Technology

Please fill the feedback form.